

Detecção de Bad Smells e Refatoração Segura

Disciplina	Testes de Software
Trabalho	Detecção de Bad Smells e Refatoração Segura
Aluno(a)	Amanda Bueno Campos Peixoto
Repositório	https://github.com/abc-peixoto/bad-smells-js-refactoring

1. Análise de Smells

O arquivo original analisado foi *src/ReportGenerator.js*. A análise manual apontou os seguintes Bad Smells principais:

- **Método Longo:** o método *generateReport* concentrava cabeçalho, corpo, regras de visibilidade, cálculo de total, prioridade e rodapé.
- **Complexidade Cognitiva Alta:** a lógica usava vários *if/else* aninhados, combinando tipo de relatório e perfil de usuário no mesmo método.
- **Duplicação de Lógica:** a montagem das linhas e a soma do total apareciam repetidas em caminhos diferentes.
- **Números Mágicos:** os limites 500 e 1000 apareciam diretamente no código, sem nomes que explicassem seu significado.

2. Relatório da Ferramenta

O projeto foi analisado com ESLint e *eslint-plugin-sonarjs*. A ferramenta apontou principalmente a complexidade cognitiva do método *generateReport*, que chegou a 27 enquanto o limite configurado era 5.

Esse resultado confirmou a análise manual: a maior dificuldade de manutenção estava no excesso de responsabilidades e condicionais dentro de um único método.

3. Processo de Refatoração

O smell mais crítico corrigido foi a alta complexidade cognitiva. A refatoração transformou *generateReport* em um método de orquestração, separando seleção de itens, cálculo de total e renderização.

Antes:

```
generateReport(reportType, user, items) {  
  let report = "";  
  let total = 0;  
  
  // cabeçalho, regras de usuário, renderização e rodapé no mesmo método  
}
```

Depois:

```
generateReport(reportType, user, items) {  
  const visibleItems = this.selectItemsForUser(user, items);
```

```

const total = this.calculateTotal(visibleItems);
const renderer = this.createRenderer(reportType, user);
return renderer.render(visibleItems, total).trim();
}

```

As principais técnicas aplicadas foram:

- **Extract Method:** as regras de seleção, cálculo de total e criação de renderizador foram separadas em métodos menores.
- **Strategy:** a renderização foi dividida em *CsvReportRenderer* e *HtmlReportRenderer*.
- **Constantes nomeadas:** os limites 500 e 1000 foram substituídos por nomes que explicam a regra de negócio.
- **Remoção de duplicação:** a montagem das linhas ficou concentrada nos renderizadores.

4. Resultados

Os prints abaixo registram a análise com ESLint, a validação do arquivo refatorado e a suíte de testes passando.

```

Round 0 vulnerabilities

C:\Users\docs\Documents\PUC\TESTES\bad-smells-js-refactoring>npx eslint src/

C:\Users\docs\Documents\PUC\TESTES\bad-smells-js-refactoring\src\ReportGenerator.js
 11:3  error  Refactor this function to reduce its Cognitive Complexity from 27 to the 5 allowed  sonarjs/cognitive-complexity
 43:14 error  Merge this if statement with the nested one                                     sonarjs/no-collapsible-if

✖ 2 problems (2 errors, 0 warnings)

```

Print 1 - Resultado do ESLint no arquivo original.

```

C:\Users\docs\Documents\PUC\TESTES\bad-smells-js-refactoring>npx eslint src/ReportGenerator.refactored.js

C:\Users\docs\Documents\PUC\TESTES\bad-smells-js-refactoring>

```

Print 2 - Validação do ESLint após a refatoração.

```

C:\Users\docs\Documents\PUC\TESTES\bad-smells-js-refactoring>npm test

> bad-smells-js-refactoring@1.0.0 test
> node --experimental-vm-modules node_modules/jest/bin/jest.js

(node:23460) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
PASS tests/ReportGenerator.test.js
PASS tests/ReportGenerator.refactored.test.js

Test Suites: 2 passed, 2 total
Tests:       10 passed, 10 total
Snapshots:   0 total
Time:        0.378 s, estimated 1 s
Ran all test suites.

```

Print 3 - Testes passando após a refatoração.

Validação final: o arquivo refatorado não apresentou erros no ESLint e a suíte completa passou com 10 testes em 2 arquivos de teste.

5. Conclusão

O uso dos testes como rede de segurança foi essencial para garantir que a refatoração preservou o comportamento esperado. A redução dos Bad Smells deixou o código mais modular, legível e fácil de modificar.